Solution of Mid Term Examination, Autumn 2025
Course Title: Microprocessors, Microcontrollers & Embedded System
Course Code: CSE-3523

---

1 (a) — 2 Marks

Explain why the **8086 uses a 20-bit address bus** and what is its **maximum addressable memory**.

The **8086 microprocessor uses a 20-bit address bus** to access memory locations.

Since the address bus has **20 lines**, the maximum address that can be generated is:

2^20 = 1,048,576 bytes

So the **maximum addressable memory** is:

1 MB (1024 KB)

This is achieved using **segment:offset addressing**, where:

Physical Address = Segment × 10H + Offset

---

1 (b) — 2 Marks

If **Physical address is 3000FH**, evaluate the **Segment and Offset address**.

Formula:

Physical Address = Segment × 10H + Offset

Given:

Physical Address = 3000FH

One possible pair is:

Segment = 3000H
Offset = 000FH

TAKEE C241014

Verification:

3000H × 10H = 30000H
30000H + 000FH = 3000FH

Therefore:

Segment Address = 3000H
Offset Address = 000FH

---

1 (c) — 6 Marks

Explain the **architectural evolution of Intel microprocessors from 8086 to Pentium**, highlighting key improvements in **architecture and performance**.

Architectural Evolution of Intel Microprocessors (8086 → Pentium)

Intel 8086 (1978)

- 16-bit processor, 20-bit address bus (1 MB memory)
- Segmented memory architecture (CS, DS, SS, ES)
- BIU (Bus Interface Unit) + EU (Execution Unit)
- 6-byte instruction prefetch queue
- No cache, no pipelining

👉 **Key Point:** Basic foundation of x86 architecture

Intel 80286 (1982/83)

- 16-bit processor, supports up to 16 MB memory
- Introduced **Protected Mode**
- Hardware memory protection and multitasking support
- Faster execution than 8086

👉 **Improvement:** Advanced memory management and OS support

Intel 80386 (1985/86)

- First **32-bit processor** (data & address bus)
- Supports up to 4 GB memory
- Introduced **paging (virtual memory)**
- Better multitasking and multi-user support

👉 **Improvement:** Transition to full 32-bit architecture

Intel 80486 (1989)

- Built-in **Floating Point Unit (FPU)**
- Integrated **L1 cache**
- **Pipelined architecture** (5-stage pipeline)
- Much faster execution

👉 **Improvement:** Significant performance boost via cache + pipelining

---

Intel Pentium (1993)

- **Superscalar architecture** (dual pipelines)
- Executes multiple instructions per clock cycle
- Separate **instruction and data cache**
- 64-bit data bus
- Improved **branch prediction**

👉 **Improvement:** Parallel processing and high-speed execution

---

Overall Architectural Improvements

- **Word Size:** 16-bit → 32-bit
- **Memory Capacity:** 1 MB → 4 GB
- **Execution:** Sequential → Pipelined → Parallel
- **Performance:** Increased from basic execution to high-speed processing
- **Integration:** Added cache, FPU, and advanced control units

2 (a) — 4 Marks

What are the **four possible fields in an assembly language statement**?
Explain the **purpose or function of each field**.

An assembly language statement can have **four possible fields**:

1. Label Field

- Used to **identify a specific instruction or memory location**
- Acts as a reference for jumps, loops, or procedures
- It is optional

Example:

```
LOOP: MOV AX, BX
```

2. Opcode (Instruction) Field

- Specifies the **operation to be performed**
- It is the actual instruction executed by the CPU

Examples:
`MOV`, `ADD`, `SUB`, `INC`

3. Operand Field

- Specifies the **data or memory location** on which the operation is performed
- Can be registers, constants, or memory addresses

Example:

```
MOV AX, BX
```

→ AX and BX are operands

4. Comment Field

- Used to **explain the instruction**
- Ignored by the assembler
- Improves readability

Example:

```
MOV AX, BX   ; Copy BX into AX
```

OR

List and explain the **status flags of 8086**.
Suppose after an arithmetic operation, the **8086 Flags Register holds the value 0257H**.
Write its **binary equivalent** and identify which **status and control flags are set (1) and reset (0)**.

The **8086 FLAGS register** contains **status flags** that indicate the result of arithmetic/logic operations.

1. Carry Flag (CF)

- Set (1) if there is a **carry out** or **borrow** in arithmetic
- Otherwise reset (0)

2. Parity Flag (PF)

- Set if the result has **even number of 1s**
- Reset if odd parity

3. Auxiliary Carry Flag (AF)

- Set if there is a carry from **lower nibble (bit 3 → bit 4)**
- Used in BCD operations

4. Zero Flag (ZF)

- Set if the result is **zero**
- Otherwise reset

5. Sign Flag (SF)

- Reflects the **sign of result**
- 1 = negative, 0 = positive

6. Overflow Flag (OF)

- Set if **signed overflow** occurs
- Otherwise reset

Given: FLAGS = 0257H

Step 1: Convert to Binary

```
0257H = 0000 0010 0101 0111 (binary)
```

Step 2: Identify Flags (bit positions)

Bit	Flag	Value
0	CF	1
2	PF	1
4	AF	1
6	ZF	1
7	SF	0
11	OF	0

Set Flags (1):

- Carry Flag (CF)
- Parity Flag (PF)
- Auxiliary Carry Flag (AF)
- Zero Flag (ZF)

Reset Flags (0):

- Sign Flag (SF)
- Overflow Flag (OF)

From 0257H, CF, PF, AF, and ZF are set, while SF and OF are reset.

2 (b) — 6 Marks

For each instruction, determine:

1. **Addressing mode**
2. **Effective address**
3. **Updated respective register and memory value**

Instructions:

```
I.   MOV AL, [BX]
II.  MOV BL,
[SI+3000H]
III. MOV CL, [BX+SI]
IV.  MOV [DI], AL
V.   MOV [BX+DI], CL
VI.  MOV AL,
[BX+DI+01H]
```

Initial contents:

```
AX = 1234H
BX = 3001H
CX = 0002H
DX = 0000H
SI = 0003H
DI = 0004H
```

Offset Address of DS	Data
3000H	22H
3001H	33H
3002H	44H
3003H	55H
3004H	66H
3005H	77H
3006H	88H
3007H	99H

Given:

- BX = 3001H, SI = 0003H, DI = 0004H
- DS memory:
  - 3000H → 22H
  - 3001H → 33H
  - 3002H → 44H
  - 3003H → 55H
  - 3004H → 66H
  - 3005H → 77H
  - 3006H → 88H
  - 3007H → 99H

I. MOV AL, [BX]

- **Addressing Mode:** Register Indirect
- **EA:** BX = 3001H
- **Operation:** AL ← [3001H] = 33H

☑ **Result:** AL = 33H

II. MOV BL, [SI+3000H]

- **Addressing Mode:** Indexed with displacement
- **EA:** SI + 3000H = 0003H + 3000H = 3003H
- **Operation:** BL ← [3003H] = 55H

☑ **Result:** BL = 55H

III. MOV CL, [BX+SI]

- **Addressing Mode:** Based Indexed
- **EA:** BX + SI = 3001H + 0003H = 3004H
- **Operation:** CL ← [3004H] = 66H

☑ **Result:** CL = 66H

IV. MOV [DI], AL

- **Addressing Mode:** Register Indirect
- **EA: DI = 0004H (Physical Address = 3004H)**
- **Operation:** [3004H] ← AL = 33H

☑ **Result:** Memory[3004H] = 33H

V. MOV [BX+DI], CL

- **Addressing Mode:** Based Indexed
- **EA:** BX + DI = 3001H + 0004H = 3005H
- **Operation:** [3005H] ← CL = 66H

☑ **Result:** Memory[3005H] = 66H

VI. MOV AL, [BX+DI+01H]

- **Addressing Mode:** Based Indexed with displacement
- **EA:** BX + DI + 01H = 3001H + 0004H + 1 = 3006H
- **Operation:** AL ← [3006H] = 88H

☑ **Result:** AL = 88H

Final Updated Values

- **AL = 88H** (updated again in step VI)
- **BL = 55H**
- **CL = 66H**

Updated Memory

- 3004H → 33H (was 66H)
- 3005H → 66H (was 77H)

3 (a) — 5 Marks

AX = **0EFFH**, BX = **125EH**

Write an **assembly language program** to:

- invert **5th and 8th bits**
- clear **rightmost 2 bits**
- set **13th and 15th bits**

After that, obtain the **final value of AX**.

Given:

AX = 0EFFH

(Binary: **0000 1110 1111 1111**)

Assembly Program

```
MOV AX, 0EFFH

; Invert 5th and 8th bits
XOR AX, 0090H      ; (bit 4 and bit 7)

; Clear rightmost 2 bits
AND AX, FFFCH

; Set 13th and 15th bits
OR AX, A000H       ; (bit 13 and bit 15)
```

Step-by-Step Result

1. Invert 5th & 8th bits

```
AX = 0EFFH XOR 0090H = 0E6FH
```

---

2. Clear rightmost 2 bits

```
AX = 0E6FH AND FFFCH = 0E6CH
```

---

3. Set 13th & 15th bits

```
AX = 0E6CH OR A000H = AE6CH
```

Final Answer: AX = AE6CH

3 (b) — 5 Marks

What will be the **final value of register and carry**?
Show **all the shifting stages**.

i)

```
MOV AL, 9CH
SHL AL, 2H
```

ii)

```
MOV AH, F7H
SAR AH, 3H
```

i)

MOV AL, 9CH

Binary:

9C = 10011100

SHL AL, 2

Step 1:

00111000

Step 2:

01110000

Final result:

AL = 70H
Carry Flag = 0

ii)

MOV AH, F7H

Binary:

F7 = 11110111

SAR AH, 3

Step 1:

11111011

Step 2:

11111101

Step 3:

11111110

Final result:

AH = FEH
Carry Flag = 1

OR

3 (b) — 5 Marks

I. For **COUNT DB 10d**, state the **variable name, memory size, and the segment where it resides**.

II. What is the purpose of the **.MODEL directive in 8086 assembly programming**?

III. What do the **.CODE and .DATA directives define in an assembly program**?

IV. What is the function of the **.STACK directive**?

V. Differentiate between **assembler directives and instructions**.

.

```
.MODEL SMALL

.STACK 100h

.DATA

COUNT DB 10d

LIMIT EQU 31

.CODE

MAIN PROC

   MOV AX,@DATA

   MOV DS,AX

   INC COUNT

MAIN ENDP

END MAIN
```

I.

Variable name: **COUNT**
Memory size: **1 byte** (because DB = Define Byte)
Segment: **DATA segment**

Explanation:
`COUNT DB 10d` defines a variable named **COUNT**, allocates **1 byte of memory**, and stores the decimal value **10** in the **data segment**.

II.

The **.MODEL directive** specifies the **memory model of the program**.
It determines how the **code, data, and stack segments are organized** in memory.

Example:

```
.MODEL SMALL
```

This means the program uses **one code segment and one data segment**.

III.

`.DATA` directive defines the **data segment** where variables and constants are stored.

Example:

```
.DATA
COUNT DB 10
```

`.CODE` directive defines the **code segment** where program instructions are written.

Example:

```
.CODE
MOV AX, BX
```

IV.

The **.STACK directive** is used to **reserve memory for the stack segment**.

Example:

```
.STACK 100h
```

This allocates **256 bytes of memory for the stack**.

The stack is used for:

- storing return addresses
- temporary data storage
- procedure calls

V.

Assembler Directives	Instructions
Used to control the assembler	Executed by the CPU
Do not generate machine code	Generate machine code
Used to define data and segments	Used to perform operations
Examples: `.DATA`, `.CODE`, `.STACK`, `DB`	Examples: `MOV`, `ADD`, `SUB`